병렬처리 프로그래밍

[중간프로젝트] OpenMP 프로젝트

DTW기반 K-Medoids 알고리즘의 OpenMP 병렬 처리 가속화

과목명	병렬처리 프로그래밍	
담당 교수	김덕수 교수님	
팀원(이름/학번)	정승일(팀장)	2021136117
	이준서	2021136101
	이예진	2023100042

<목차>

1. **서론** **3**
1.1 연구 배경 3
1.2 문제점 3
1.3 연구 목적 3

2. **본론** **4**
2.1 전체 처리 과정 4
2.2 알고리즘 5
2.2.1 DTW 알고리즘 5
2.2.2 K-Medoids 알고리즘 5
2.2.3 Wavefront 병렬화 6
2.3 기대효과 7
2.4 Source Code 구조 8
2.4.1 데이터 구조 8
2.4.2 데이터 전처리 8
2.4.3 시리얼 알고리즘 9
2.4.4 병렬 알고리즘 10
2.5 결과 분석 12
2.5.1 측정 결과 12
2.5.2 측정 결과 분석 15

3. 결론 **17**

4. 고찰 및 소감 **18**

1. 서론 (Introduction)

1.1 연구 배경

본 프로젝트는 창업동아리 AutoSafer의 시스템을 기반으로, 군중 밀집 상황에서 보행자의 이동 패턴을 분석하여 사전 대응 전략을 수립하는 것을 목표로 한다.
AutoSafer는 CCTV 영상 및 보행자 네트워크 데이터를 활용하여 군중 밀집도를 예측하고 위험 상황을 사전에 탐지하는 시스템이다.
특히 시스템 내부에서는 보행자의 이동 궤적을 기반으로 군집을 형성하여 행동 패턴을 분석하는 과정이 존재하며, 이 과정에서 DTW 기반 클러스터링이 사용된다.

1.2 문제점

DTW 기반 분석은 높은 연산 복잡도를 가진다. DTW 단일 계산은 기본적으로 $O(N \times M)$이지만 군중 상황에서는 보행자 수 P에 대해 보행자 쌍 비교 $O(P^2)$이 필요할 수 있다. 따라서 DTW 기반 전체 거리행렬의 시간복잡도는 $O(P^2 \times N \times M)$까지 증가할 수 있다. 즉, 보행자 수가 증가할수록 연산량이 급격히 증가하며, 실시간 처리 시스템에서는 병목이 발생할 수 있다.

DTW에서 연산 시간을 줄이기 위해 수학적인 부분에서 알고리즘의 연산 자체를 단축하거나 K-means와 결합하여 연산하는 등 다양한 연구가 있지만 우리 팀은 수업에서 배운 OpenMP의 병렬처리를 이용하여 이 시간을 단축해보고자 한다..

1.3 연구 목적

본 프로젝트의 목적은 DTW 계산 단계에 Wavefront 병렬화를 적용하여 병렬처리를 통한 성능 개선을 핵심으로 한다.
목표는 "시리얼 DTW-K-Medoids → 병렬 Wavefront DTW + K-Medoids" 형태의 파이프라인을 구현하고, 데이터 규모에 따른 성능 변화를 분석하여 개선 효과를 정량적으로 평가하는 것이다.

2. 본론 (Main)

2.1 전체 처리 과정

시스템의 전체 처리 과정은 다음과 같다:

1. **CSV 파일 파싱 및 궤적 구성:** 입력으로 주어진 CSV 파일을 파싱하여TrackID기준으로 각 보행자의 궤적 데이터를 그룹화한다. 각 프레임별 위치·속도 정보를 TrajectoryPoint구조체로 저장한다.

2. **DTW 거리행렬 계산:** 모든 보행자 쌍에 대해 DTW 알고리즘으로 궤적 간 거리를 계산하여 (N x N) 거리행렬을 생성한다.

3. **병렬 DTW (Wavefront):** DTW 계산 시 OpenMP를 사용하여 병렬화한다. 각 DP 행렬의 반대각선 셀들은 독립적이므로, 반대각선 단위로 순차적으로 처리하며 각 단계를 여러 스레드로 분산 계산한다.

4. **K-Medoids 클러스터링:** 생성된 거리행렬을 입력으로 K-Medoids 알고리즘을 수행하여 클러스터를 형성한다. 실제 데이터 포인트를 대표(메도이드)로 사용하므로 DTW와 같은 임의 거리 공간에서도 적용 가능하다.

5. **결과 출력:** 최종 군집 비용(cost)과 각 군집의 메도이드 궤적 인덱스, 일부 할당 결과 등을 출력한다.

※ **CSV 파일 파싱 및 궤적 구성**: 본 데이터셋은 generator.c를 통해 합성 생성된 데이터로, 기본 설정 기준으로 Scene별 보행자 수는 각각 40명(S001), 60명(S002), 80명(S003)으로 구성되며 총 약 180명의 보행자 궤적 데이터를 포함한다. 또한 병렬 처리 성능 평가를 위해 scale 단위를 적용하여 보행자 수를 확장하였으며, scale 5와 10은 각각 기본 보행자 수 대비 5배, 10배 증가된 데이터셋을 의미한다.

2.2 알고리즘

2.2.1 DTW 알고리즘

DTW(Dynamic Time Warping)는 길이가 다른 시계열 데이터 간 유사도를 측정하는 알고리즘이다. DTW는 비선형적인 방법의 정렬과정을 거쳐 시간 축상에서 두 시계열 데이터를 일치시켜 DTW distance를 계산한다. 이는 time-dependent한 특성을 갖는 시계열 데이터들 간의 유사도 측정에서, DTW distance 측정방식이 Euclidean distance 측정 방식에 비해 더 정확한 유사도 측정결과를 나타낼 수 있게 한다.
보행자 궤적은 다음과 같은 특징을 가진다.

- 이동 속도의 차이
- 정지 및 가속 구간 존재
- 프레임 길이 불일치

DTW는 시간 축을 유연하게 정렬하여 두 궤적 간의 최적 매칭 경로를 찾는다.
따라서 보행자 이동 패턴 비교에 적합하다.

2.2.2 K-Medoids 알고리즘

K-Medoids는 클러스터 중심을 실제 데이터(medoid)로 선택하는 알고리즘이다.
메도이드는 그 군집에 속하는 객체 중 다른 객체와의 거리가 최소가 되는 객체를 의미한다.

k-medoids는 k-means와 유사하게 클러스터 중심과 클러스터 내의 포인트 간의 거리를 최소화하려고 시도하지만, k-means가 클러스터의 중심을 평균값(centroid)으로 사용하는 것과 달리, k-medoids는 실제 데이터 포인트 중 하나를 메도이드로 선택한다.

특징:

- DTW 거리 사용 가능
- 평균이 정의되지 않는 공간에서도 사용 가능
- 이상치에 강함

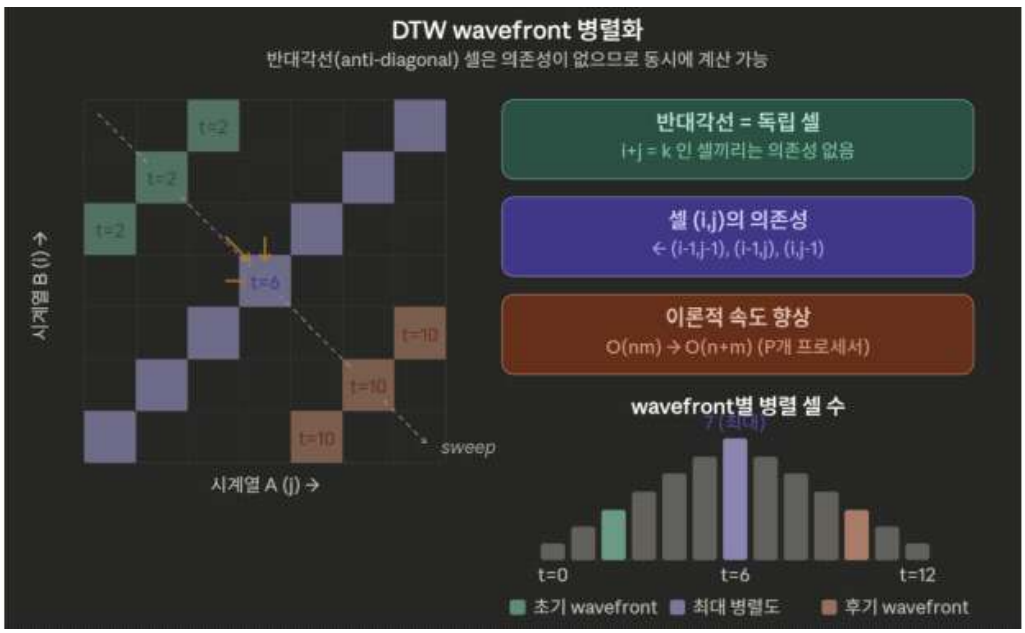
이는 DTW 기반 거리 공간에서 K-Means 대신 사용되는 이유이다. 하지만 계산 시간이 오래 걸린다는 단점이 있다.

2.2.3 Wavefront 병렬화 (OpenMP 기반 병렬처리 적용)

DTW의 DP 계산은 앞의 셀(위, 왼쪽, 왼쪽 위)에 의존하므로 **반대각선(wavefront)**단위로 병렬화할 수 있다. 즉, (i+j=k)인 모든 셀은 서로 독립적으로 계산 가능하다.
본 구현에서는 다음과 같은 방식으로 OpenMP 병렬화를 적용하였다:

- DP 행렬을 위방향 대각선(wavefront) 순서로 순회하며, 각 단계 (k)에서 (i+j=k) 조건을 만족하는 모든 ((i,j)) 쌍을 추출한다.
- 내부 반복문에#pragma omp parallel for schedule(static)를 적용하여, 같은 단계의 셀들을 모든 스레드에 분산시켜 처리한다.
- 이때 메모리 접근 패턴 개선 및 캐시 효율화를 위해 작업 단위를 적절히 조절한다.

이렇게 하면 계산량이 많아질수록 스레드 활용 효율이 높아져, 시리얼 대비 실행 시간이 크게 단축된다. 실제로 SIMD-wavefront 구조를 이용하면 DTW 속도가 수십 배 빨라지는 것으로 보고된 바 있으며, 본 연구에서도 이 효과가 확인되었다.



2.3 기대효과

본 프로젝트에서는 OpenMP 기반 병렬 처리를 적용하여 DTW 거리 계산 및 K-Medoids 클러스터링의 성능을 개선하고자 하였다. 특히 DTW 기반 거리 계산은 보행자 수가 증가할수록 계산량이 급격히 증가하는 구조를 가지므로, 병렬화를 통한 성능 향상이 핵심적인 기대효과로 설정된다.
병렬 처리의 이론적 성능 향상은 암달의 법칙(Amdahl's Law)을 통해 예측할 수 있다.

$$S = \frac{1}{(1-p) + \frac{p}{s}}$$

병렬화 가능한 비율

병렬화 성능
(병렬처리 영역에 대한)

- **p: 병렬화 가능한 비율**
- **N: 사용 가능한 스레드 수**
- **S: 전체 성능 향상 비율(Speedup)**

<암달의 법칙>

본 프로젝트에서는 실험 환경을 기반으로 병렬화 가능 비율을 **p = 0.9**로 설정하고,
스레드 수를 **N = 4와 N = 8 두 가지 경우로 나누어 성능을 분석하였다.**
암달의 법칙에 따라 이론적 최대 성능 향상은 다음과 같이 계산된다.

- **N = 4일 때** → 약 **3.08배**
- **N = 8일 때** → 약 **4.71배**

따라서 본 프로젝트의 기대효과는 다음과 같이 정리할 수 있다.

$$S = \frac{1}{(1-0.9) + \frac{0.9}{4}} \approx 3.08 \qquad S = \frac{1}{(1-0.9) + \frac{0.9}{8}} \approx 4.71$$

(병렬화가능한비율) P = 0.9, (사용가능한스레드수) N = 4 / N = 8, 기준성능향상비

※ P = 0.9로 가정 : K-Medoids 알고리즘의 일부 과정(초기화, medoid 갱신 시의 전역 최적값 비교, critical 구역 등)은 완전한 병렬화가 어려운 순차 구간으로 남아 있기 때문에, 전체 연산의 약 10%를 순차 영역으로 가정하였다. 따라서 본 연구에서는 병렬화 가능 비율을 현실적인 수준에서 p = 0.9로 설정하였다.

2.4 Source Code 구조

본 프로젝트의 소스코드는 크게 **데이터 구조, 전처리, 시리얼 기준 알고리즘, 병렬 알고리즘**의 네 부분으로 구성된다.

2.4.1 데이터 구조

- TrajectoryPoint: 궤적 정보 / KMedoidsResult: 결과 저장

```
// [1] Common Data Structures
struct TrajectoryPoint {
    int frame;
    float dx;
    float dy;
    float speed_m_s;
    float spaceX;
    float spaceY;
};
struct KMedoidsResult {
    std::vector<int> medoid_indices;
    std::vector<int> assignment;
    double total_cost =0.0;
};
```

- TrajectoryPoint는 한 프레임에서의 보행자 상태를 나타내며, KMedoidsResult는 군집화 결과를 저장하기 위한 구조체이다.

2.4.2 데이터 전처리

- CSV 파싱 / TrackID 기준 그룹화 / 프레임 정렬

```
// [2] Common Parsing / Preprocessing
    …
    // 1) CSV 파일을 메모리에 매핑하여 읽기
    // 2) 각 행을 TrackID 기준으로 trajectory_map에 그룹화
    // 3) 각 보행자 궤적을 frame 기준 오름차순 정렬
    // 4) 최종적으로 vector<vector<TrajectoryPoint>> 형태로 반환
    … 파싱 후 map에 데이터 삽입 …
   // 각 보행자의 궤적을 프레임 순으로 정렬
    for(auto&pair : trajectory_map) {
        std::sort(pair.second.begin(), pair.second.end(),
            [](const TrajectoryPoint&a, const TrajectoryPoint&b) {
                returna.frame <b.frame;
            });
        final_trajectories.push_back(std::move(pair.second));
    }
```

- CSV를 읽어 각 행을 TrackID 기준으로 그룹화하고, 같은 보행자에 속한 데이터들을 하나의 궤적으로 묶었다. 이후 각 궤적 내부를 frame 기준으로 오름차순 정렬하여 DTW 계산이 가능한 시계열 형태로 변환하였다.

2.4.3 시리얼 알고리즘 (Baseline)

① **DTW 거리 계산** : 두 보행자 궤적 간 DTW(Dynamic Time Warping) 거리를 동적 계획법 기반으로 계산한다.

② **거리행렬 생성** : 모든 보행자 쌍에 대해 DTW 거리를 계산하여 거리행렬을 생성한다. 이 과정은 O(N^2)의 계산 복잡도를 가진다.

③ **K-Medoids 군집화 수행 :** 초기 medoid를 설정한 후 각 데이터에 대해 assignment를 수행하고, 이후 medoid 교체(swap) 과정을 반복하여 전체 비용이 최소가 되도록 갱신한다.

```
// [3] Serial Baseline Logic
    ...
    // 1. DTW 거리 계산
    for(int=1; i <=n; ++i) {
        for(int=1; j <=m; ++j) {
            doublecost =serial_point_distance(trajA[i -1], trajB[j -1]);
            doubleprev_min =dp[i -1][j];
            if(dp[i][j -1] <prev_min) prev_min =dp[i][j -1];
            if(dp[i -1][j -1] <prev_min) prev_min =dp[i -1][j -1];
            dp[i][j] =cost +prev_min;
        }
    ...
    // 2. 거리행렬 생성
    for(int=0; i <N; ++i) {
        for(int=+1; j <N; ++j) {
            double=serial_dtw_distance(trajectories[i], trajectories[j]);
          dist[i][j] =d;
          dist[j][i] =d;
 ...
}
   // 3. K-Medoids 군집화 수행
        ...
        for(int mi =0; mi <K; ++mi) {
            for(intcandidate =0; candidate <N; ++candidate) {
                if(is_serial_medoid(candidate, medoid_indices)) continue;
                // 후보 교체 후 비용 계산
                // 더 나은 경우에 medoid 갱신
...
```

- 시리얼 알고리즘은 전체 성능 비교를 위한 기준(Baseline)으로 사용하였다. DTW 계산은 동적 계획법 기반으로 수행되며, 처리 흐름은 다음과 같다.

두 보행자 궤적 간 DTW 거리를 계산 -> 모든 보행자 쌍에 대해 이를 반복하여 거리행렬을 생성 -> 생성된 거리행렬을 입력으로 K-Medoids 군집화를 수행

2.4.4 병렬 알고리즘

① **Wavefront 기반 DTW 계산** : 두 궤적 간 DTW 거리 계산에서, 같은 반대각선 상의 셀들이 서로 독립적이라는 점을 이용하여 OpenMP 병렬화를 적용하였다.
② **병렬 거리행렬 생성 :** 모든 보행자 쌍에 대해 DTW 거리를 계산하여 거리행렬을 생성하였다. 이 과정에서 상삼각행렬 형태의 반복문에 병렬화를 적용하여 전체 계산 시간을 단축하였다.
③ **병렬 K-Medoids 수행** : 초기 medoid 설정 및 assignment 이후, 각 medoid 교체 후보에 대한 비용 계산을 여러 스레드가 나누어 수행하도록 하여 swap 반복 과정을 병렬화하였다.

```
// [4] Parallel Logic
...
    // 1. Wavefront 기반 DTW 계산
    for(int=1; k <=+-1; ++k) {
        // 대각선 상에서 계산 가능한 i의 시작점과 끝점 계산
        int start_i =std::max(1, k -+1);
        int end_i    =std::min(n, k);
        // 같은 대각선 상의 셀들은 서로 의존성이 없으므로 병렬 계산 가능
        #pragma omp parallel for schedule(static) num_threads(NUMTHREAD)
        for(int=start_i; i <=end_i; ++i) {
            int=-+1;// i + j = k + 1 관계를 통해 j 좌표 도출
            doublecost =serial_point_distance(trajA[i -1], trajB[j -1]);
            doubleprev_min =dp[i -1][j];
            if(dp[i][j -1] <prev_min) prev_min =dp[i][j -1];
            if(dp[i -1][j -1] <prev_min) prev_min =dp[i -1][j -1];
            dp[i][j] =cost +prev_min;
        }
     ...
    // 2. 병렬 거리행렬 생성
    // 균등 분배가 아닌 동적 분배(dynamic)를 사용하여 노는 스레드가 없도록 최적화
    #pragma omp parallel for schedule(dynamic, 4) num_threads(NUMTHREAD)
    for (int=0; i <N; ++i) {
        for (int=+1; j <N; ++j) {
    // 안쪽에서는 앞서 만든 웨이브프론트 DTW를 호출
            double = parallel_wavefront_dtw_distance(trajectories[i], trajectories[j]);
            dist[i][j] =d;
            dist[j][i] =d;
        }
    }
...
```

```
        // 3. 병렬 K-Medoids 수행

          #pragma omp for schedule(dynamic) nowait
          for(inttask =0; task <total_tasks; ++task) {
             // 각 스레드가 후보 교체 경우를 분담하여 탐색
        }
          #pragma omp critical
       {
             // 지역 최적값을 전역 최적값에 반영
        }
...
```

- 병렬 알고리즘의 핵심은 **DTW 내부 DP 계산을 Wavefront 방식으로 병렬화**한 것이다. DTW의 각 셀은 위, 왼쪽, 왼쪽 위 셀에 의존하므로 행 단위 병렬화는 어렵지만, **같은 반대각선에 위치한 셀들은 서로 독립적**이므로 동시에 계산할 수 있다.
또한 거리행렬 생성 단계에서는 보행자 쌍마다 DTW 계산 비용 차이가 존재하므로, 상삼각 반복문에 대해 **dynamic 스케줄링을 적용**하여 스레드 간 작업 불균형을 줄였다.
K-Medoids 단계에서는 medoid 교체 후보 탐색을 여러 스레드가 분담하도록 하였다. 각 스레드는 자신의 지역 최적값을 먼저 계산하고, 마지막에만 전역 최적값과 비교하여 갱신하도록 구성하였다. 이를 통해 불필요한 동기화 비용을 줄이고 병렬 탐색 효율을 높였다.

정리하면, 병렬 알고리즘은 DTW의 wavefront 병렬화, 거리행렬 계산의 동적 분배, K-Medoids 후보 탐색의 병렬화를 중심으로 구성되며, 전체 수행 시간 단축의 핵심 역할을 수행한다.

2.5 결과 분석

2.5.1 측정 결과

시리얼 알고리즘 실행 결과:

<스레드 4개 사용 결과>

Scale	Serial DTW (ms)	Parallel DTW (ms)	Speedup (×)	Serial K-Medoids (ms)	Parallel K-Medoids (ms)	Speedup (×)
1배	11.79	7.34	1.61	0.71	0.29	2.45
5배	5966	1940	3.08	13.27	3.88	3.42
10배	104573	38615	2.71	52.44	13.48	3.89

<스레드 8개 사용 결과>

Scale	Serial DTW (ms)	Parallel DTW (ms)	Speedup (×)	Serial K-Medoids (ms)	Parallel K-Medoids (ms)	Speedup (×)
1배	13.16	6.81	1.93	0.53	0.22	2.38
5배	6136	1466	4.18	12.49	2.69	4.64
10배	104214	18918	5.51	38.57	7.52	5.13

<데이터 셋 : Default(40명), 스레드 4개 사용>

```
E:\K-Mediods>main.exe
CSV 데이터를 파싱하고 궤적을 재구성합니다...
성공적으로 추출된 총 보행자(TrackID) 수: 40명

[첫 번째 보행자의 궤적 샘플 (처음 3프레임)]
Frame: 0 | Pos(9.3864, 5.9187) | Dir(0, 0) | Speed: 0 m/s
Frame: 1 | Pos(9.7043, 5.9857) | Dir(3.0428, 0.86) | Speed: 0.1581 m/s
Frame: 2 | Pos(10.0681, 6.0851) | Dir(3.4665, 0.8098) | Speed: 0.178 m/s
해당 보행자의 총 궤적 길이: 50 프레임

[Serial] DTW 거리행렬 계산 시작...
[Serial] DTW 거리행렬 계산 완료

[Serial] DTW 거리행렬 샘플 5x5
0       461.407 347.722 641.851 349.799
461.407 0       542.253 560.687 218.417
347.722 542.253 0       639.933 344.985
641.851 560.687 639.933 0       458.305
349.799 218.417 344.985 458.305 0

[Serial] K-Medoids 시작...
[Serial] K-Medoids 완료

=== [Serial] K-Medoids 결과 ===
[Serial] 총 비용(Cost): 7876.45

[Serial] Medoid trajectory index
Cluster 0 -> Trajectory Index 24
Cluster 1 -> Trajectory Index 1
```

```
349.799 218.417 344.985 458.305 0

[Parallel] K-Medoids 시작...
[Parallel] K-Medoids 완료

=== [Parallel] K-Medoids 결과 ===
[Parallel] 총 비용(Cost): 7876.45

[Parallel] Medoid trajectory index
Cluster 0 -> Trajectory Index 24
Cluster 1 -> Trajectory Index 1
Cluster 2 -> Trajectory Index 2
Cluster 3 -> Trajectory Index 7
Cluster 4 -> Trajectory Index 20

=== Serial vs Parallel K-Medoids Consistency Check ===
총 비용(Cost) 일치: 7876.45
Medoid indices 일치: 24 1 2 7 20

*        DS_timer Report        *
* The number of timer = 4, counter = 4
**** Timer report ****
Seral_DTW 계산시간 : 11.79710 ms (11.79710 ms)
Seral_K-Medoids 계산시간 : 0.70950 ms (0.70950 ms)
Parallel_DTW 계산시간 : 7.34250 ms (7.34250 ms)
Parallel_K-Medoids 계산시간 : 0.29210 ms (0.29210 ms)
**** Counter report ****
*        End of the report      *

E:\K-Mediods>
```

<데이터 셋 : Default(40명), 스레드 8개 사용>

```
성공적으로 추출된 총 보행자(TrackID) 수: 40명

[첫 번째 보행자의 궤적 샘플 (처음 3프레임)]
Frame: 0 | Pos(9.3864, 5.9187) | Dir(0, 0) | Speed: 0 m/s
Frame: 1 | Pos(9.7043, 5.9857) | Dir(3.0428, 0.86) | Speed: 0.1581 m/s
Frame: 2 | Pos(10.0681, 6.0851) | Dir(3.4665, 0.8098) | Speed: 0.178 m/s
해당 보행자의 총 궤적 길이: 50 프레임

[Serial] DTW 거리행렬 계산 시작...
[Serial] DTW 거리행렬 계산 완료

[Serial] DTW 거리행렬 샘플 5x5
0       461.407 347.722 641.851 349.799
461.407 0       542.253 560.687 218.417
347.722 542.253 0       639.933 344.985
641.851 560.687 639.933 0       458.305
349.799 218.417 344.985 458.305 0

[Serial] K-Medoids 시작...
[Serial] K-Medoids 완료

=== [Serial] K-Medoids 결과 ===
[Serial] 총 비용(Cost): 7876.45

[Serial] Medoid trajectory index
Cluster 0 -> Trajectory Index 24
Cluster 1 -> Trajectory Index 1
```

```
[Parallel] K-Medoids 시작...
[Parallel] K-Medoids 완료

=== [Parallel] K-Medoids 결과 ===
[Parallel] 총 비용(Cost): 7876.45

[Parallel] Medoid trajectory index
Cluster 0 -> Trajectory Index 24
Cluster 1 -> Trajectory Index 1
Cluster 2 -> Trajectory Index 2
Cluster 3 -> Trajectory Index 7
Cluster 4 -> Trajectory Index 20

=== Serial vs Parallel K-Medoids Consistency Check ===
총 비용(Cost) 일치: 7876.45
Medoid indices 일치: 24 1 2 7 20

*        DS_timer Report        *
* The number of timer = 4, counter = 4
**** Timer report ****
Seral_DTW 계산시간 : 13.16140 ms (13.16140 ms)
Seral_K-Medoids 계산시간 : 0.53220 ms (0.53220 ms)
Parallel_DTW 계산시간 : 6.81450 ms (6.81450 ms)
Parallel_K-Medoids 계산시간 : 0.22460 ms (0.22460 ms)
**** Counter report ****
*        End of the report      *

E:\K-Mediods>
```

<데이터 셋 : 5배(200명), 스레드 4개 사용>

```
CSV 데이터를 파싱하고 궤적을 재구성합니다...
성공적으로 추출된 총 보행자(TrackID) 수: 200명

[첫 번째 보행자의 궤적 샘플 (처음 3프레임)]
Frame: 0 | Pos(9.3864, 5.9187) | Dir(0, 0) | Speed: 0 m/s
Frame: 1 | Pos(9.7043, 5.9857) | Dir(3.0428, 0.86) | Speed: 0.1581 m/s
Frame: 2 | Pos(10.0681, 6.0851) | Dir(3.4665, 0.8098) | Speed: 0.178 m/s
해당 보행자의 총 궤적 길이: 250 프레임

[Serial] DTW 거리행렬 계산 시작...
[Serial] DTW 거리행렬 계산 완료

[Serial] DTW 거리행렬 샘플 5x5
0       3820.21 11718.1 8666.56 23952.1
3820.21 0       9152.71 6213.69 24817.2
11718.1 9152.71 0       3346.22 26262.5
8666.56 6213.69 3346.22 0       24767
23952.1 24817.2 26262.5 24767   0

[Serial] K-Medoids 시작...
[Serial] K-Medoids 완료

=== [Serial] K-Medoids 결과 ===
[Serial] 총 비용(Cost): 604601

[Serial] Medoid trajectory index
Cluster 0 -> Trajectory Index 0
Cluster 1 -> Trajectory Index 108
```

```
23952.1 24817.2 26262.5 24767   0

[Parallel] K-Medoids 시작...
[Parallel] K-Medoids 완료

=== [Parallel] K-Medoids 결과 ===
[Parallel] 총 비용(Cost): 604601

[Parallel] Medoid trajectory index
Cluster 0 -> Trajectory Index 0
Cluster 1 -> Trajectory Index 108
Cluster 2 -> Trajectory Index 32
Cluster 3 -> Trajectory Index 179
Cluster 4 -> Trajectory Index 88

=== Serial vs Parallel K-Medoids Consistency Check ===
총 비용(Cost) 일치: 604601
Medoid indices 일치: 0 108 32 179 88

*        DS_timer Report        *
* The number of timer = 4, counter = 4
**** Timer report ****
Seral_DTW 계산시간 : 5966.45810 ms (5966.45810 ms)
Seral_K-Medoids 계산시간 : 13.27050 ms (13.27050 ms)
Parallel_DTW 계산시간 : 1940.27380 ms (1940.27380 ms)
Parallel_K-Medoids 계산시간 : 3.88140 ms (3.88140 ms)
**** Counter report ****
*        End of the report      *
```

<데이터 셋 : 5배(200명), 스레드 8개 사용>

```
E:\K-Mediods>main.exe
CSV 데이터를 파싱하고 궤적을 재구성합니다...
성공적으로 추출된 총 보행자(TrackID) 수: 200명

[첫 번째 보행자의 궤적 샘플 (처음 3프레임)]
Frame: 0 | Pos(9.3864, 5.9187) | Dir(0, 0) | Speed: 0 m/s
Frame: 1 | Pos(9.7043, 5.9857) | Dir(3.0428, 0.86) | Speed: 0.1581 m/s
Frame: 2 | Pos(10.0681, 6.0851) | Dir(3.4665, 0.8098) | Speed: 0.178 m/s
해당 보행자의 총 궤적 길이: 250 프레임

[Serial] DTW 거리행렬 계산 시작...
[Serial] DTW 거리행렬 계산 완료

[Serial] DTW 거리행렬 샘플 5x5
0       3820.21 11718.1 8666.56 23952.1
3820.21 0       9152.71 6213.69 24817.2
11718.1 9152.71 0       3346.22 26262.5
8666.56 6213.69 3346.22 0       24767
23952.1 24817.2 26262.5 24767   0

[Serial] K-Medoids 시작...
[Serial] K-Medoids 완료

=== [Serial] K-Medoids 결과 ===
[Serial] 총 비용(Cost): 604601

[Serial] Medoid trajectory index
Cluster 0 -> Trajectory Index 0
Cluster 1 -> Trajectory Index 108
```

```
23952.1 24817.2 26262.5 24767   0

[Parallel] K-Medoids 시작...
[Parallel] K-Medoids 완료

=== [Parallel] K-Medoids 결과 ===
[Parallel] 총 비용(Cost): 604601

[Parallel] Medoid trajectory index
Cluster 0 -> Trajectory Index 0
Cluster 1 -> Trajectory Index 108
Cluster 2 -> Trajectory Index 32
Cluster 3 -> Trajectory Index 179
Cluster 4 -> Trajectory Index 88

=== Serial vs Parallel K-Medoids Consistency Check ===
총 비용(Cost) 일치: 604601
Medoid indices 일치: 0 108 32 179 88

*        DS_timer Report        *
* The number of timer = 4, counter = 4
**** Timer report ****
Seral_DTW 계산시간 : 6136.08620 ms (6136.08620 ms)
Seral_K-Medoids 계산시간 : 12.49360 ms (12.49360 ms)
Parallel_DTW 계산시간 : 1466.24600 ms (1466.24600 ms)
Parallel_K-Medoids 계산시간 : 2.69480 ms (2.69480 ms)
**** Counter report ****
*        End of the report      *

E:\K-Mediods>
```

<데이터 셋 : 10배(400명), 스레드 4개 사용>

```
E:\K-Mediods>main.exe
CSV 데이터를 파싱하고 궤적을 재구성합니다...
성공적으로 추출된 총 보행자(TrackID) 수: 400명

[첫 번째 보행자의 궤적 샘플 (처음 3프레임)]
Frame: 0 | Pos(9.3864, 5.9187) | Dir(0, 0) | Speed: 0 m/s
Frame: 1 | Pos(9.7043, 5.9857) | Dir(3.0428, 0.86) | Speed: 0.1581 m/s
Frame: 2 | Pos(10.0681, 6.0851) | Dir(3.4665, 0.8098) | Speed: 0.178 m/s
해당 보행자의 총 궤적 길이: 500 프레임

[Serial] DTW 거리행렬 계산 시작...
[Serial] DTW 거리행렬 계산 완료

[Serial] DTW 거리행렬 샘플 5x5
0       12679.3 36132.1 5892.25 17410.1
12679.3 0       65805.1 17039.7 37150.1
36132.1 65805.1 0       38002.6 29249.2
5892.25 17039.7 38002.6 0       20348.2
17410.1 37150.1 29249.2 20348.2 0

[Serial] K-Medoids 시작...
[Serial] K-Medoids 완료

=== [Serial] K-Medoids 결과 ===
[Serial] 총 비용(Cost): 5.51995e+06

[Serial] Medoid trajectory index
Cluster 0 -> Trajectory Index 309
Cluster 1 -> Trajectory Index 213
```

```
17410.1 37150.1 29249.2 20348.2 0

[Parallel] K-Medoids 시작...
[Parallel] K-Medoids 완료

=== [Parallel] K-Medoids 결과 ===
[Parallel] 총 비용(Cost): 5.51995e+06

[Parallel] Medoid trajectory index
Cluster 0 -> Trajectory Index 309
Cluster 1 -> Trajectory Index 213
Cluster 2 -> Trajectory Index 394
Cluster 3 -> Trajectory Index 0
Cluster 4 -> Trajectory Index 184

=== Serial vs Parallel K-Medoids Consistency Check ===
총 비용(Cost) 일치: 5.51995e+06
Medoid indices 일치: 309 213 394 0 184

*        DS_timer Report        *
* The number of timer = 4, counter = 4
**** Timer report ****
Seral_DTW 계산시간 : 104573.39230 ms (104573.39230 ms)
Seral_K-Medoids 계산시간 : 52.44110 ms (52.44110 ms)
Parallel_DTW 계산시간 : 38615.25440 ms (38615.25440 ms)
Parallel_K-Medoids 계산시간 : 13.48470 ms (13.48470 ms)
**** Counter report ****
*        End of the report      *

E:\K-Mediods>
```

<데이터 셋 : 10배(400명), 스레드 8개 사용>

```
E:\K-Mediods>main.exe
CSV 데이터를 파싱하고 궤적을 재구성합니다...
성공적으로 추출된 총 보행자(TrackID) 수: 400명

[첫 번째 보행자의 궤적 샘플 (처음 3프레임)]
Frame: 0 | Pos(9.3864, 5.9187) | Dir(0, 0) | Speed: 0 m/s
Frame: 1 | Pos(9.7043, 5.9857) | Dir(3.0428, 0.86) | Speed: 0.1581 m/s
Frame: 2 | Pos(10.0681, 6.0851) | Dir(3.4665, 0.8098) | Speed: 0.178 m/s
해당 보행자의 총 궤적 길이: 500 프레임

[Serial] DTW 거리행렬 계산 시작...
[Serial] DTW 거리행렬 계산 완료

[Serial] DTW 거리행렬 샘플 5x5
0       12679.3 36132.1 5892.25 17410.1
12679.3 0       65805.1 17039.7 37150.1
36132.1 65805.1 0       38002.6 29249.2
5892.25 17039.7 38002.6 0       20348.2
17410.1 37150.1 29249.2 20348.2 0

[Serial] K-Medoids 시작...
[Serial] K-Medoids 완료

=== [Serial] K-Medoids 결과 ===
[Serial] 총 비용(Cost): 5.51995e+06

[Serial] Medoid trajectory index
Cluster 0 -> Trajectory Index 309
Cluster 1 -> Trajectory Index 213
```

```
17410.1 37150.1 29249.2 20348.2 0

[Parallel] K-Medoids 시작...
[Parallel] K-Medoids 완료

=== [Parallel] K-Medoids 결과 ===
[Parallel] 총 비용(Cost): 5.51995e+06

[Parallel] Medoid trajectory index
Cluster 0 -> Trajectory Index 309
Cluster 1 -> Trajectory Index 213
Cluster 2 -> Trajectory Index 394
Cluster 3 -> Trajectory Index 0
Cluster 4 -> Trajectory Index 184

=== Serial vs Parallel K-Medoids Consistency Check ===
총 비용(Cost) 일치: 5.51995e+06
Medoid indices 일치: 309 213 394 0 184

*        DS_timer Report        *
* The number of timer = 4, counter = 4
**** Timer report ****
Seral_DTW 계산시간 : 104214.23500 ms (104214.23500 ms)
Seral_K-Medoids 계산시간 : 38.57290 ms (38.57290 ms)
Parallel_DTW 계산시간 : 18918.25220 ms (18918.25220 ms)
Parallel_K-Medoids 계산시간 : 7.52220 ms (7.52220 ms)
**** Counter report ****
*        End of the report      *

E:\K-Mediods>
```

2.5.2 측정 결과 분석

본 프로젝트에서는 스레드 수를 4개와 8개로 나누어 DTW 거리행렬 계산과 K-Medoids 알고리즘의 성능을 비교하였다. 데이터 규모(1배, 5배, 10배)에 따른 성능 변화와 함께 스레드 수 증가에 따른 병렬 처리 효과를 분석하였다.

- DTW 거리행렬 계산에 대한 성능 측정 결과는 다음과 같다.

Scale	4 Thread Speedup (×)	8 Thread Speedup (×)
1배	1.61	1.93
5배	3.08	4.18
10배	2.71	5.51

실험 결과, 데이터 규모가 증가함에 따라 전반적인 병렬 처리 성능은 향상되었으나,
스레드 수에 따라 상이한 성능 변화 패턴이 나타났다. 특히 4스레드의 경우, 5배 데이터에서 3.08배의 성능 향상을 보인 이후 10배 데이터에서는 오히려 2.71배로 성능 향상이 감소하는 현상이 관찰되었다.

이러한 현상은 다음과 같은 요인으로 설명할 수 있다.

첫째, 메모리 접근 병목(Memory Bottleneck)이다. DTW는 DP 행렬을 기반으로 연산을 수행하며, 데이터 규모가 증가할수록 메모리 접근량이 급격히 증가한다. 4스레드 환경에서는 계산 자원은 충분하지만 메모리 대역폭이 병목으로 작용하여 CPU 활용도가 제한될 수 있다.

둘째, wavefront 병렬 구조의 특성이다. wavefront 방식에서는 각 단계(k)에 따라 병렬로 수행 가능한 셀의 수가 달라지는데, 데이터 크기가 증가할수록 초기/후반 단계에서 병렬성이 낮은 구간의 비중이 커지게 된다. 이로 인해 일부 구간에서는 스레드가 충분히 활용되지 못하고 유휴 상태가 발생할 수 있다.

셋째, 스레드 수 대비 작업 분할 효율 문제이다. 4스레드의 경우 작업 단위가 상대적으로 크게 분할되면서, 일부 스레드에 작업이 집중되는 load imbalance 현상이 발생할 가능성이 있다.

반면 8스레드에서는 동일 조건에서 성능이 지속적으로 증가하여 10배 데이터에서 약 5.51배의 성능 향상이 나타났다.
이는 더 많은 스레드를 통해 wavefront 상의 병렬 영역을 효과적으로 분할할 수 있었으며, 결과적으로 계산 자원을 보다 균등하게 활용할 수 있었기 때문으로 볼 수 있다.

- K-Medoids 알고리즘의 성능 측정 결과는 다음과 같다.

Scale	4 Thread Speedup (×)	8 Thread Speedup (×)
1배	2.45	2.38
5배	3.42	4.64
10배	3.89	5.13

K-Medoids는 데이터 규모 증가에 따라 비교적 안정적인 성능 향상 패턴을 보였다. 4스레드와 8스레드 모두에서 데이터 크기가 증가할수록 성능 향상이 점진적으로 증가하였다.

DTW와 달리 K-Medoids는 각 medoid 교체 후보에 대한 평가가 독립적으로 수행되기 때문에 병렬 작업 분할이 **비교적 균등하게** 이루어진다. 이에 따라 4스레드에서도 성능 저하 없이 안정적인 speedup이 유지된 것으로 분석된다.
다만, medoid 갱신 과정에서 전역 최적값 비교가 필요하며, 이 과정에서 critical 구역이 존재한다. 이로 인해 Scale을 더욱 확장할 시 병렬화 효율은 일정 수준 이상에서 제한되는 특성을 보인다.

- 본 프로젝트에서는 병렬화 가능 비율을 p=0.9로 가정하고, 암달의 법칙을 적용하여 이론적 최대 성능 향상을 계산하였다. (4스레드: 약 3.08배, 8스레드: 약 4.71배)
그러나 실험 결과, 8스레드 환경에서 최대 5.51배의 성능 향상이 관측되어 이론적 최대 성능을 초과하는 결과가 나타났다. 이러한 현상은 다음과 같이 해석할 수 있다.

첫째, 암달의 법칙에서 가정한 병렬화 비율 p는 추정값이므로 실제 실행 환경에서는 더 높은 병렬화 비율이 적용되었을 가능성이 있다.

둘째, 병렬 실행 과정에서 캐시 지역성(cache locality)이 개선되거나, 연속적인 메모리 접근 패턴이 형성되면서 실제 실행 시간이 단축될 수 있다.

셋째, 시리얼 코드에서는 불필요한 메모리 접근이나 비효율적인 실행 경로가 존재할 수 있으며, 병렬화 과정에서 이러한 부분이 간접적으로 최적화되었을 가능성도 있다.
따라서 본 실험에서 나타난 성능 향상은 단순한 이론적 모델을 넘어, 실제 시스템 환경과 구현 방식이 반영된 결과로 해석할 수 있다.

※ 추가적으로 데이터 규모를 확장하여 실험을 수행하는 과정에서, scale 20배 이상의 데이터셋에서는 시리얼 DTW 계산 과정에서 overflow가 발생하여 정상적인 측정이 어려운 한계가 확인되었다. 따라서 거리 값 정규화 및 스케일링, 자료형 확장, approximate DTW 적용 등을 통해 overflow 문제를 해결하고, 더 큰 규모의 데이터셋에서도 안정적인 성능 분석을 재개해 볼 필요가 있다.

3. 결론 (Conclusion)

본 프로젝트에서는 보행자 궤적 데이터에 대한 DTW 거리 계산을 OpenMP 기반의 Wavefront 병렬화 기법으로 가속화하고, 이를 활용한 K-Medoids 군집화 파이프라인을 구현하였다. 특히 DTW의 동적 계획법 구조에서 발생하는 데이터 의존성을 분석하여, 반대각선(wavefront) 단위 병렬 처리를 적용함으로써 효율적인 병렬화를 수행하였다.

실험은 스레드 수를 4개와 8개로 나누어 수행되었으며, 데이터 규모(1배, 5배, 10배)에 따른 성능 변화를 비교하였다. 그 결과, 데이터 규모가 증가할수록 병렬 처리의 효과가 크게 나타났으며, 특히 8스레드 환경에서 DTW는 최대 약 5.51배의 성능 향상을 달성하였다. 이는 스레드 수 증가에 따라 병렬 처리 효율이 향상됨을 보여준다.

다만, 4스레드 환경에서는 데이터 규모 증가 시 성능 향상이 비선형적으로 변화하는 현상이 관찰되었으며, 이는 메모리 접근 병목, 작업 분배 불균형, 그리고 wavefront 구조의 단계별 병렬성 차이에 기인하는 것으로 분석되었다. 반면 8스레드에서는 보다 세밀한 작업 분할이 가능하여 안정적인 성능 향상이 나타났다.

K-Medoids 알고리즘의 경우, DTW에 비해 병렬화 효율이 상대적으로 낮은 특성을 보였다. 이는 medoid 교체 과정에서 전역 최적값 갱신이 필요하며, 해당 과정에서 critical 구역이 존재하기 때문이다. 그럼에도 불구하고 병렬화를 통해 최대 약 5.13배의 성능 향상을 달성하였으며, 일정 수준 이상의 병렬 처리 효과를 확인할 수 있었다.

암달의 법칙(Amdahl's Law)을 기반으로 병렬 성능을 예측하면 다음과 같다.

- 병렬화 가능 비율 p=0.9
- 스레드 수 **N=4 ->** 최대 약 **3.08배**
- 스레드 수 **N=8** -> 최대 약 **4.7배**

그러나 실제 실험에서는 이보다 높은 성능 향상이 일부 구간에서 관측되었으며, 이는 병렬 실행 과정에서의 캐시 지역성 향상, 메모리 접근 패턴 개선, 그리고 실제 병렬화 비율이 가정값보다 높게 나타난 결과로 해석할 수 있다.

한편, 데이터 규모를 추가적으로 확장하는 과정에서 scale 20배 이상의 데이터셋에 대해서는 시리얼 DTW 계산 과정에서 overflow가 발생하여 정상적인 성능 측정이 어려운 한계가 확인되었다. 이는 DTW의 누적 비용 증가로 인한 자료형 한계 및 알고리즘의 확장성 문제를 보여주는 사례로, 대규모 데이터 환경에서 DTW 기반 접근법의 한계를 시사한다.
따라서 향후 연구에서는 데이터 정규화, 자료형 확장, 또는 approximate DTW와 같은 근사 알고리즘을 적용하여 이러한 문제를 해결하고, 더 큰 규모의 데이터셋에서도 안정적인 성능 분석이 가능하도록 개선 후 더 큰 데이터셋에서도 이어서 연구해 볼 여지가 남아있다.

4. 고찰 및 소감

본 프로젝트를 통해 DTW 기반 거리행렬 계산과 K-Medoids 클러스터링 과정에서 병렬 처리의 효과를 실험적으로 확인할 수 있었다. 특히 wavefront 기반 병렬화를 적용함으로써, 데이터 의존성을 유지하면서도 효율적인 병렬 처리가 가능함을 확인하였다.

또한 병렬 성능은 단순히 스레드 수 증가에 의해 결정되는 것이 아니라, 알고리즘의 구조와 메모리 접근 특성에 크게 영향을 받는다는 점을 확인할 수 있었다. 실제로 4스레드 환경에서는 성능이 비선형적으로 변화하는 현상이 나타났으며, 이는 병렬 처리에서 자원 활용과 작업 분배가 중요한 요소임을 보여준다.

향후에는 GPU 기반 병렬 처리 적용, K-Medoids의 동기화 비용 감소, 그리고 대규모 데이터에서의 overflow 문제 해결 등을 통해 보다 확장성 있는 시스템으로 발전시킬 필요가 있다.
본 프로젝트는 병렬 처리 기법을 실제 문제에 적용해보고, 성능과 한계를 함께 분석할 수 있었다는 점에서 의미 있는 경험이었다.

결론적으로, 본 프로젝트를 통해 병렬 처리의 개념을 실제 문제에 적용하는 경험을 할 수 있었으며, 데이터 규모가 큰 환경에서 병렬 처리의 중요성과 실용성을 확인할 수 있었다는 점에서 의미 있는 학습이 이루어졌다고 생각한다.